



Como o banco de dados executa consultas e quais elementos influenciam a performance

Prof. Gustavo Dias

Ao longo da disciplina de Banco de Dados, foi estudado como modelar informações utilizando o modelo Entidade-Relacionamento, convertê-las para o modelo relacional e implementar fisicamente as tabelas em um Sistema Gerenciador de Banco de Dados (SGBD), como o MySQL. Entretanto, escrever uma consulta SQL correta não significa necessariamente escrever uma consulta eficiente.

Em sistemas reais, bancos de dados podem armazenar milhões ou bilhões de registros. Nesse contexto, pequenas diferenças na estrutura do banco ou na forma como uma consulta é escrita podem representar diferenças de desempenho de alguns milissegundos para vários minutos. Por esse motivo, compreender como o SGBD executa uma consulta é um dos conhecimentos mais importantes para um profissional de Sistemas de Informação.

Segundo Silberschatz, Korth e Sudarshan (2020), o objetivo de um SGBD moderno é fornecer acesso eficiente aos dados, utilizando mecanismos internos de

armazenamento, indexação e otimização de consultas para reduzir o custo de processamento.

1. Como o SGBD executa uma consulta SQL

Quando um comando SQL é enviado ao banco de dados, o SGBD não executa imediatamente a instrução exatamente da forma como ela foi escrita. O processo envolve diversas etapas internas.

Fluxo simplificado:

Consulta SQL → Parser → Otimizador → Plano de execução → Acesso aos dados → Resultado

O **parser** verifica a sintaxe da consulta e sua validade. Em seguida, o **otimizador de consultas** analisa diferentes estratégias para executar o comando e escolhe aquela com menor custo estimado. Essa estratégia escolhida é chamada de **plano de execução**.

Por exemplo:

```
SELECT * FROM cliente  
WHERE cpf = '12345678900';
```

O banco pode decidir:

- utilizar um índice sobre a coluna cpf;
- percorrer toda a tabela (full table scan);
- combinar índices diferentes;
- reordenar operações de junção (JOINS).

O programador especifica **o que deseja**, enquanto o SGBD decide **como obter o resultado** (ELMASRI; NAVATHE, 2017).

2. Índices: o principal mecanismo de otimização

O que é um índice?

Um índice é uma estrutura de dados criada para acelerar a localização de registros em uma tabela. Sua função é semelhante ao índice de um livro: em vez de ler todas as páginas, é possível ir diretamente ao conteúdo desejado.

Exemplo:

```
CREATE INDEX idx_cliente_cpf  
ON cliente(cpf);
```

Após a criação do índice, consultas utilizando a coluna cpf podem ser executadas muito mais rapidamente.

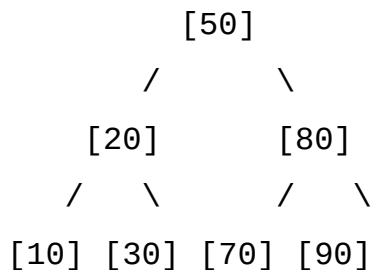
Estrutura B-tree

O mecanismo de armazenamento InnoDB do MySQL utiliza predominantemente **B-tree (árvore balanceada)** para implementar índices.

Características principais:

- busca em tempo aproximadamente logarítmico;
- inserções e remoções eficientes;
- boa performance para consultas de igualdade e intervalos.

De forma simplificada:



A busca por um valor não percorre todos os registros da tabela, mas apenas um pequeno conjunto de páginas da árvore.

Segundo Schwartz, Zaitsev e Tkachenko (2012), índices são frequentemente responsáveis pelos maiores ganhos de desempenho em bancos relacionais.

Índice primário e secundário

- **Índice primário:** associado à chave primária da tabela.
- **Índice secundário:** criado sobre outras colunas.

Exemplo:

```
CREATE INDEX idx_nome  
ON cliente(nome);
```

Índices compostos

Um índice pode envolver mais de uma coluna.

```
CREATE INDEX idx_cidade_idade  
ON cliente(cidade, idade);
```

Esse índice pode ser utilizado eficientemente para:

```
WHERE cidade = 'Poços de Caldas'
```

ou

```
WHERE cidade = 'Poços de Caldas'
```

```
    AND idade = 30
```

Entretanto, normalmente não será utilizado para:

```
WHERE idade = 30
```

Esse comportamento decorre da regra do **prefixo à esquerda** (leftmost prefix), adotada pelo MySQL.

Quando um índice não ajuda?

Índices não são uma solução universal.

Situações em que podem ter pouco benefício:

- colunas com poucos valores distintos (sexo, status, ativo/inativo);
- tabelas muito pequenas;
- consultas que retornam grande parte dos registros.

Além disso, índices aumentam o custo de operações de:

- INSERT;
- UPDATE;
- DELETE.

Portanto, deve existir equilíbrio entre desempenho de leitura e custo de manutenção.

3. EXPLAIN e o plano de execução

Para compreender como o MySQL executará uma consulta, utiliza-se o comando **EXPLAIN**.

Exemplo:

```
EXPLAIN
```

```
SELECT *
```

```
FROM cliente
```

```
WHERE cpf = '12345678900';
```

O resultado apresenta informações como:

- tabela acessada;
- índice utilizado;
- quantidade estimada de linhas;
- tipo de acesso;
- operações adicionais.

Alguns tipos importantes:

- **const** – acesso extremamente eficiente;
- **ref** – busca por índice;
- **range** – intervalo em índice;
- **ALL** – varredura completa da tabela.

Uma consulta que utiliza **ALL** em uma tabela com milhões de registros geralmente merece investigação.

A documentação oficial do MySQL recomenda o uso frequente do EXPLAIN durante o desenvolvimento e otimização de consultas (MYSQL, 2025).

4. Estatísticas e o otimizador de consultas

O otimizador decide qual plano utilizar com base em **estatísticas** armazenadas pelo banco de dados.

Essas estatísticas incluem informações como:

- quantidade de registros;

- distribuição dos valores;
- cardinalidade dos índices;
- seletividade das colunas.

Cardinalidade

É o número de valores distintos existentes em uma coluna.

Exemplo:

Tabela com 1.000.000 de registros.

Coluna cpf: 1.000.000 valores distintos.

Coluna sexo: apenas 2 valores distintos.

Índices sobre colunas de alta cardinalidade tendem a ser muito mais eficientes.

Seletividade

Representa a capacidade de um índice reduzir o conjunto de registros analisados.

Quanto maior a seletividade, maior a probabilidade de o índice ser utilizado.

5. O catálogo de dados (Data Dictionary)

Todo SGBD mantém um conjunto de metadados conhecido como **catálogo de dados** ou **dicionário de dados**.

Esses metadados descrevem:

- tabelas;
- colunas;
- tipos de dados;
- índices;
- restrições;
- permissões;
- estatísticas.

No MySQL, os metadados podem ser consultados principalmente através do **INFORMATION_SCHEMA**.

Exemplo:

```
SELECT table_name, table_rows  
FROM information_schema.tables  
WHERE table_schema = 'hospital';
```

Também é possível consultar colunas:

```
SELECT column_name, data_type  
FROM information_schema.columns  
WHERE table_name = 'cliente';
```

O catálogo de dados é utilizado por administradores, desenvolvedores, ferramentas de modelagem e pelo próprio otimizador de consultas.

Segundo Elmasri e Navathe (2017), o dicionário de dados é um componente essencial de qualquer SGBD relacional.

6. Restrições de integridade e desempenho

As restrições (constraints) garantem consistência dos dados e podem auxiliar o otimizador.

Principais restrições:

- PRIMARY KEY;
- FOREIGN KEY;
- UNIQUE;
- NOT NULL;
- CHECK.

Exemplo:

```
CREATE TABLE cliente (  
    id INT PRIMARY KEY,  
    cpf VARCHAR(11) UNIQUE,  
    nome VARCHAR(100) NOT NULL  
);
```

A chave primária gera automaticamente um índice no InnoDB, enquanto a restrição UNIQUE normalmente também cria uma estrutura indexada.

Essas informações ajudam o banco a eliminar possibilidades durante a execução da consulta.

7. Views (Visualizações)

Uma **view** é uma consulta armazenada que pode ser utilizada como se fosse uma tabela.

Exemplo:

```
CREATE VIEW vw_clientes_ativos AS  
SELECT id, nome  
FROM cliente  
WHERE ativo = 1;
```

Depois:

```
SELECT *  
FROM vw_clientes_ativos;
```

Vantagens

- simplificação de consultas complexas;
- reutilização de lógica;
- abstração da estrutura física;
- controle de acesso aos dados.

Views melhoram desempenho?

Em geral, **não**.

No MySQL, uma view normalmente é expandida durante o processamento da consulta, sendo tratada como a consulta original.

Em outros SGBDs existem **materialized views**, que armazenam fisicamente os resultados e podem oferecer ganhos significativos em consultas analíticas.

8. Triggers

Uma **trigger** é um procedimento executado automaticamente quando ocorre um evento em uma tabela.

Eventos principais:

- INSERT;
- UPDATE;
- DELETE.

Exemplo:

```
CREATE TRIGGER trg_log_cliente  
AFTER INSERT ON cliente  
FOR EACH ROW  
INSERT INTO log_cliente (cliente_id, data_evento)  
VALUES (NEW.id, NOW());
```

Sempre que um cliente for inserido, um registro será gravado na tabela de log.

Aplicações comuns

- auditoria;
- histórico de alterações;
- validações;
- sincronização entre tabelas;
- regras de negócio automáticas.

Impacto na performance

Triggers adicionam processamento extra às operações de escrita.

Um simples INSERT pode passar a executar diversas operações adicionais, aumentando o tempo da transação.

Por isso, seu uso deve ser criterioso, principalmente em sistemas com grande volume de atualizações.

9. Normalização, JOINS e custo de consultas

A normalização reduz redundância e melhora a consistência dos dados.

Entretanto, modelos altamente normalizados frequentemente exigem múltiplos JOINS.

Cada JOIN representa trabalho adicional para o SGBD.

Em sistemas analíticos (Data Warehouse), muitas vezes utiliza-se **desnormalização controlada** para reduzir a quantidade de junções necessárias.

Segundo Kimball e Ross (2013), ambientes de consulta intensiva frequentemente privilegiam modelos que favorecem leitura rápida em detrimento da completa normalização.

10. Boas práticas iniciais para consultas SQL eficientes

Antes de tentar otimizar uma consulta, verifique:

- Existe índice nas colunas do WHERE?
- Os JOINS utilizam colunas indexadas?
- Estou aplicando funções sobre colunas indexadas?
- A consulta retorna apenas as colunas necessárias?
- Há filtros suficientes para reduzir o conjunto de dados?
- O EXPLAIN indica uso de índice?
- Existem triggers ou views adicionando processamento extra?

Exemplo de prática inadequada:

```
SELECT *  
FROM cliente;
```

Melhor abordagem:

```
SELECT id, nome  
FROM cliente  
WHERE ativo = 1;
```

Evitar SELECT * reduz leitura de páginas, tráfego de rede e uso de memória.

Conclusão

O desempenho de uma consulta SQL depende muito mais do funcionamento interno do SGBD do que da sintaxe utilizada. Índices, estatísticas, planos de execução, catálogo

de dados, restrições, views e triggers formam um conjunto de mecanismos que influencia diretamente o custo de processamento de cada comando SQL.

Compreender esses elementos é o primeiro passo para escrever consultas escaláveis e preparar-se para técnicas avançadas de otimização, análise de desempenho e administração de bancos de dados relacionais.

Referências

ELMASRI, R.; NAVATHE, S. B. *Fundamentals of Database Systems*. 7. ed. Boston: Pearson, 2017.

KIMBALL, R.; ROSS, M. *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*. 3. ed. Hoboken: John Wiley & Sons, 2013.

MYSQL. *MySQL 8.4 Reference Manual*. Oracle Corporation, 2025. Disponível em: <https://dev.mysql.com/doc/refman/8.4/en/>. Acesso em: 9 ago. 2026.

SCHWARTZ, B.; ZAITSEV, P.; TKACHENKO, V. *High Performance MySQL: Optimization, Backups, and Replication*. 3. ed. Sebastopol: O'Reilly Media, 2012.

SILBERSCHATZ, A.; KORTH, H. F.; SUDARSHAN, S. *Database System Concepts*. 7. ed. New York: McGraw-Hill, 2020.